Connections to Calculus

From the Chain Rule to ChatGPT: The Mathematics Behind AI

John Mitchell · Clark College · WAMATYC 2026

May 2026

The Calculus in the Story

Large language models like GPT-3 are trained by solving an optimization problem with 175 billion variables. The algorithm that solves it — gradient descent via backpropagation — is an application of single-variable and multivariable calculus. Every step your students learn in the calculus sequence has a direct counterpart in modern AI.

The table below maps the AI concept to the calculus course where it appears. The rest of this handout fills in the details.

AI concept	**Calculus connection**
Derivative of loss function	Calc 1: derivatives, chain rule
Gradient descent update rule	Calc 1: tangent line, decreasing functions
Backpropagation	Calc 1: chain rule (repeated)
Sigmoid, ReLU derivatives	Calc 1: quotient rule, piecewise functions
Loss as a sum	Calc 2: summation, accumulation
Gradient descent as a sequence	Calc 3: sequences and convergence
Forward pass as vector map	Calc 3: vector-valued functions
Partial derivatives of loss	Calc 4: partial derivatives
Gradient vector ∇L	Calc 4: gradient, directional derivative
Computation graph	Calc 4: multivariable chain rule, dependency diagrams
Gradient descent (full)	Calc 4: optimization of multivariable functions

Derivatives Your Students Already Know

Each of the following appears directly in neural network training. All are standard Calc 1 results.

The Sigmoid and Its Derivative

The sigmoid activation function is:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Its derivative is one of the most elegant results in applied calculus:

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$$

Derivation. Write $\sigma(z) = (1 + e^{-z})^{-1}$ and apply the chain rule:

$$\sigma'(z) = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2}$$

Rewrite $e^{-z} = (1 + e^{-z}) - 1$ and factor:

$$= \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z)(1 - \sigma(z))$$

This is a direct application of the chain rule and algebra. No new tools required.

The fact that σ' is expressible in terms of σ itself is computationally important: during backpropagation, the forward pass has already computed $\sigma(z)$, so the derivative costs nothing extra.

The Cross-Entropy Gradient

For binary classification with prediction $\hat{y}$ and true label $y \in \{0, 1\}$, the cross-entropy loss is:

$$L = -\big[y \log \hat{y} + (1 - y) \log(1 - \hat{y})\big]$$

Differentiating with respect to $\hat{y}$:

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$$

When combined with the sigmoid derivative in the chain rule, the $\hat{y}(1 - \hat{y})$ terms cancel, leaving the remarkably clean result:

$$\frac{\partial L}{\partial z} = \hat{y} - y$$

The gradient of the loss with respect to the pre-activation input is simply the prediction error. This is not a coincidence — it is a consequence of the mathematical compatibility between the sigmoid activation and the cross-entropy loss.

The Chain Rule at Scale

From One Variable to a Computation Graph

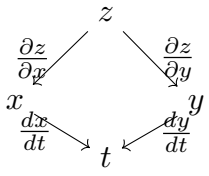
Your students learn the chain rule as: if $y = f(g(x))$, then

$$\frac{dy}{dx} = f'(g(x)) \cdot g'(x)$$

In Calc 4, this extends to functions of several variables. If $z = f(x, y)$ where $x = x(t)$ and $y = y(t)$, then:

$$\frac{dz}{dt} = \frac{\partial f}{\partial x}\frac{dx}{dt} + \frac{\partial f}{\partial y}\frac{dy}{dt}$$

Calculus instructors often draw a *dependency diagram* to track which variables depend on which:



This diagram *is* a computation graph. The only difference between a Calc 4 dependency diagram and a neural network computation graph is scale: a neural network has millions of nodes rather than four, and the chain rule is applied along every path from every parameter to the loss. The structure is identical.

Key result. Backpropagation is the multivariable chain rule applied to a computation graph. Every dependency diagram your Calc 4 students draw is a miniature version of what happens inside a neural network during training. The only thing that changes at scale is the number of nodes — not the mathematics.

Historical note. The chain rule was developed independently by Leibniz and Newton in the 1670s–1680s as part of the foundations of calculus. The multivariable generalization emerged in the 18th and 19th centuries through the work of Euler, Lagrange, and Cauchy. Neither could have anticipated that three centuries later, the same rule would be the computational engine running inside systems processing billions of words of text. The chain rule is not a historical artifact of the calculus curriculum — it is an active tool in the most computationally intensive systems humans have ever built.

Student Activity: Backpropagation by Hand

Background

A neural network learns by adjusting its parameters to reduce prediction error. The adjustment rule is:

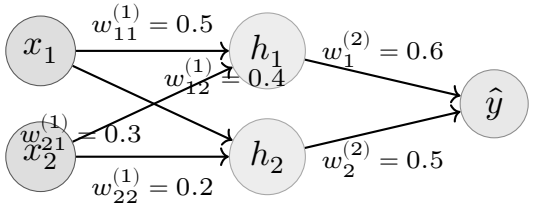
$$\theta \leftarrow \theta - \alpha \cdot \frac{\partial L}{\partial \theta}$$

where L is the loss (how wrong the prediction is), α is the learning rate (step size), and $\frac{\partial L}{\partial \theta}$ tells us how much L changes when we nudge θ.

Computing $\frac{\partial L}{\partial \theta}$ for every parameter in the network is called **backpropagation**. It is an application of the chain rule, applied layer by layer from the output back to the inputs. In this activity you will do it by hand on a small network.

The Network

We use a $2 \to 2 \to 1$ network: 2 input features, 2 hidden neurons, 1 output neuron. The activation function is the sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$ throughout.



Initial weights and biases (all biases set to zero for clarity):

Layer 1 weights		**Layer 2 weights**	
$w_{11}^{(1)} = 0.5$	$w_{12}^{(1)} = 0.4$	$w_1^{(2)} = 0.6$	$w_2^{(2)} = 0.5$
$w_{21}^{(1)} = 0.3$	$w_{22}^{(1)} = 0.2$		

One training example: input $(x_1, x_2) = (1.0,\ 0.5)$, target $y = 1$.

Learning rate: $\alpha = 0.5$.

Loss function: Mean squared error, $L = \frac{1}{2}(\hat{y} - y)^2$. (The $\frac{1}{2}$ is a convenience that cancels the exponent when differentiating.)

Part 1: The Forward Pass

Compute the pre-activation and activation at each node.

Hidden neuron h_1:

$$z_1^{(1)} = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 = (0.5)(1.0) + (0.4)(0.5) = _____$$

$$h_1 = \sigma(z_1^{(1)}) = _____$$

Hidden neuron h_2:

$$z_2^{(1)} = w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 = (0.3)(1.0) + (0.2)(0.5) = _____$$

$$h_2 = \sigma(z_2^{(1)}) = _____$$

Output neuron:

$$z^{(2)} = w_1^{(2)} h_1 + w_2^{(2)} h_2 = _____$$

$$\hat{y} = \sigma(z^{(2)}) = _____$$

Loss:

$$L = \tfrac{1}{2}(\hat{y} - 1)^2 = _____$$

Part 2: The Backward Pass

Apply the chain rule layer by layer, moving from output to input.

Step 1 — Gradient of loss with respect to output:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = _____$$

Step 2 — Gradient through the output sigmoid (recall $\sigma'(z) = \sigma(z)(1-\sigma(z))$):

$$\frac{\partial L}{\partial z^{(2)}} = \frac{\partial L}{\partial \hat{y}} \cdot \sigma'(z^{(2)}) = \frac{\partial L}{\partial \hat{y}} \cdot \hat{y}(1-\hat{y}) = _____$$

Step 3 — Gradients for Layer 2 weights (let $\delta^{(2)} = \frac{\partial L}{\partial z^{(2)}}$):

$$\frac{\partial L}{\partial w_1^{(2)}} = \delta^{(2)} \cdot h_1 = _____ \qquad \frac{\partial L}{\partial w_2^{(2)}} = \delta^{(2)} \cdot h_2 = _____$$

Step 4 — Propagate gradient to hidden layer. The chain rule extends the path one layer further. For hidden neuron h_1:

$$\frac{\partial L}{\partial z_1^{(1)}} = \delta^{(2)} \cdot w_1^{(2)} \cdot \sigma'(z_1^{(1)}) = \delta^{(2)} \cdot w_1^{(2)} \cdot h_1(1-h_1) = _____$$

Define $\delta_1^{(1)} = \frac{\partial L}{\partial z_1^{(1)}}$. Similarly for h_2:

$$\delta_2^{(1)} = \delta^{(2)} \cdot w_2^{(2)} \cdot h_2(1-h_2) = _____$$

Step 5 — Gradients for Layer 1 weights:

$$\frac{\partial L}{\partial w_{11}^{(1)}} = \delta_1^{(1)} \cdot x_1 = _____ \qquad \frac{\partial L}{\partial w_{12}^{(1)}} = \delta_1^{(1)} \cdot x_2 = _____$$

$$\frac{\partial L}{\partial w_{21}^{(1)}} = \delta_2^{(1)} \cdot x_1 = _____ \qquad \frac{\partial L}{\partial w_{22}^{(1)}} = \delta_2^{(1)} \cdot x_2 = _____$$

Part 3: Weight Updates

Apply the gradient descent update $\theta \leftarrow \theta - \alpha \cdot \frac{\partial L}{\partial \theta}$ with $\alpha = 0.5$ to each weight. Fill in the updated values:

Weight	Old value	Gradient	New value
$w_{11}^{(1)}$	0.5		
$w_{12}^{(1)}$	0.4		
$w_{21}^{(1)}$	0.3		
$w_{22}^{(1)}$	0.2		
$w_1^{(2)}$	0.6		
$w_2^{(2)}$	0.5		

What just happened. You computed 6 partial derivatives of a function of 6 variables, evaluated at a specific point, using only the chain rule applied twice. This is backpropagation. In GPT-3, the same procedure runs on a function of 175 billion variables, thousands of times per second, on specialized hardware. The mathematics is unchanged.

Answer Key

Forward pass:

$z_1^{(1)} = 0.70, \quad h_1 = \sigma(0.70) \approx 0.6682$

$z_2^{(1)} = 0.40, \quad h_2 = \sigma(0.40) \approx 0.5987$

$z^{(2)} = (0.6)(0.6682) + (0.5)(0.5987) \approx 0.7002$

$\hat{y} = \sigma(0.7002) \approx 0.6682$

$L = \frac{1}{2}(0.6682 - 1)^2 \approx 0.0549$

Backward pass:

$\frac{\partial L}{\partial \hat{y}} = 0.6682 - 1 = -0.3318$

$\delta^{(2)} = (-0.3318)(0.6682)(1 - 0.6682) \approx -0.0735$

$\frac{\partial L}{\partial w_1^{(2)}} = (-0.0735)(0.6682) \approx -0.0491$

$\frac{\partial L}{\partial w_2^{(2)}} = (-0.0735)(0.5987) \approx -0.0440$

$\delta_1^{(1)} = (-0.0735)(0.6)(0.6682)(1 - 0.6682) \approx -0.0147$

$\delta_2^{(1)} = (-0.0735)(0.5)(0.5987)(1 - 0.5987) \approx -0.0088$

$\frac{\partial L}{\partial w_{11}^{(1)}} = (-0.0147)(1.0) \approx -0.0147$

$\frac{\partial L}{\partial w_{12}^{(1)}} = (-0.0147)(0.5) \approx -0.0074$

$\frac{\partial L}{\partial w_{21}^{(1)}} = (-0.0088)(1.0) \approx -0.0088$

$\frac{\partial L}{\partial w_{22}^{(1)}} = (-0.0088)(0.5) \approx -0.0044$

Updated weights ($\alpha = 0.5$):

Weight	**Old**	**Gradient**	**New**
$w_{11}^{(1)}$	0.5000	-0.0147	0.5074
$w_{12}^{(1)}$	0.4000	-0.0074	0.4037
$w_{21}^{(1)}$	0.3000	-0.0088	0.3044
$w_{22}^{(1)}$	0.2000	-0.0044	0.2022
$w_1^{(2)}$	0.6000	-0.0491	0.6246
$w_2^{(2)}$	0.5000	-0.0440	0.5220

All weights increased slightly, moving $\hat{y}$ closer to the target $y = 1$. The Layer 2 weights moved more than the Layer 1 weights — because the gradient signal attenuates as it passes back through each sigmoid derivative, a phenomenon called *vanishing gradients*.

Connections Across the Calculus Sequence

Calculus 1: Derivatives and the Chain Rule

Backpropagation is Calc 1. The forward pass is a composition of functions; the backward pass differentiates that composition using the chain rule. Every gradient in the activity above required nothing beyond what appears in a first-quarter calculus course.

Specific connections:

- Derivative of e^x and the quotient rule $\rightarrow$ sigmoid derivative
- Chain rule for $\frac{d}{dx}f(g(x)) \rightarrow$ one step of backprop
- Tangent line approximation $\rightarrow$ gradient descent update (step in the direction of steepest descent)
- Increasing/decreasing functions $\rightarrow$ the sign of the gradient tells us which direction to move

Framing for students: "The chain rule is not just a technique for computing derivatives. It is the algorithm that trains every neural network on the planet. When you learn to differentiate a composition, you are learning the mathematics of machine intelligence.''

Calculus 2: Summation and Accumulation

The connection to Calc 2 is less direct but real. Loss functions are averages over training examples:

$$L = \frac{1}{n}\sum_{i=1}^{n} \ell(\hat{y}_i, y_i)$$

The summation notation, the idea of accumulating a quantity over many cases, and the passage from a finite sum to a limit — these are Calc 2 themes. The convergence of gradient descent (does the sequence of losses $L_0, L_1, L_2, ...$ approach zero?) is a convergence question that connects to series.

A richer connection in Calc 2. The cross-entropy loss $-\log p$ has an information-theoretic interpretation rooted in the natural logarithm. The derivative $\frac{d}{dp}(-\log p) = -1/p$ is a Calc 1 result, but the reason this loss is preferred over MSE for classification — its connection to maximum likelihood estimation — involves the logarithm as an accumulator of log-probabilities, a theme that resonates with the Calc 2 treatment of $\ln x$.

Calculus 3: Sequences, Vectors, and Convergence

Gradient descent produces a sequence of parameter vectors:

$$\theta_0,\ \theta_1,\ \theta_2,\ ...$$

Each term is computed from the previous: $\theta_{t+1} = \theta_t - \alpha \nabla L(\theta_t)$. This is a recursively defined sequence — a central object in Calc 3. Questions about convergence (does the sequence reach a fixed point? how fast?) are sequence questions.

The forward pass through a neural network is a composition of vector-valued functions. Each layer maps $\mathbb{R}^n \to \mathbb{R}^m$: input vector in, output vector out. The full network is:

$$\mathbb{R}^{d_{\text{in}}} \xrightarrow{f^{(1)}} \mathbb{R}^{d_1} \xrightarrow{f^{(2)}} \cdots \xrightarrow{f^{(N)}} \mathbb{R}^{d_{\text{out}}}$$

This is a chain of vector-valued functions — the object Calc 3 students encounter in the context of parametric curves and space curves, now applied to information processing.

Calculus 4: Partial Derivatives, Gradients, and the Chain Rule

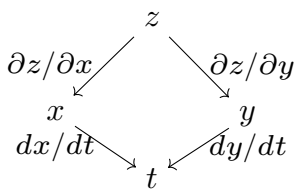
This is where the connection is deepest. Neural network training is optimization of a scalar-valued function of many variables:

$$\min_{\theta \in \mathbb{R}^n} L(\theta)$$

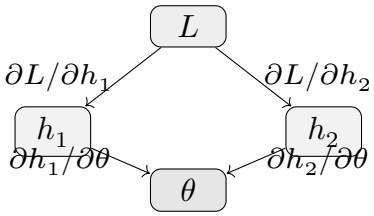
For GPT-3, $n = 175{,}000{,}000{,}000$. The tools are exactly those of Calc 4:

- **Partial derivatives** $\frac{\partial L}{\partial \theta_i}$ — each gradient computation in backprop is a partial derivative
- **The gradient vector** $\nabla L = \left(\frac{\partial L}{\partial \theta_1}, \ldots, \frac{\partial L}{\partial \theta_n}\right)$ — this is what backprop computes
- **Gradient descent** $\theta \leftarrow \theta - \alpha \nabla L$ — steepest descent in $\mathbb{R}^n$, a direct generalization of the single-variable case
- **The multivariable chain rule** — the computation graph *is* a dependency diagram

The dependency diagram connection. The multivariable chain rule is typically illustrated in Calc 4 with a tree diagram showing how a variable depends on intermediate variables. This diagram is identical in structure to a neural network computation graph. When your students draw a dependency diagram for $\frac{dz}{dt}$ where $z = f(x, y)$, $x = x(t)$, $y = y(t)$, they are drawing a two-layer neural network with one output. The chain rule they apply to evaluate $\frac{dz}{dt}$ is exactly the algorithm that trains GPT-3. The only difference is scale.



Calc 4 dependency diagram



Computation graph

The structures are the same. The mathematics is the same. The scale is different by a factor of roughly 40 billion.